Assignment -14.1

Hallticket no:2303A510A6

Batch no:02

Task 1 – Image Gallery Website

Create an image gallery web page.

Requirements:

- **Display images in a grid layout.**
- **Use AI to:**

o Optimize layout using CSS Grid.

o Add hover zoom or overlay effects.

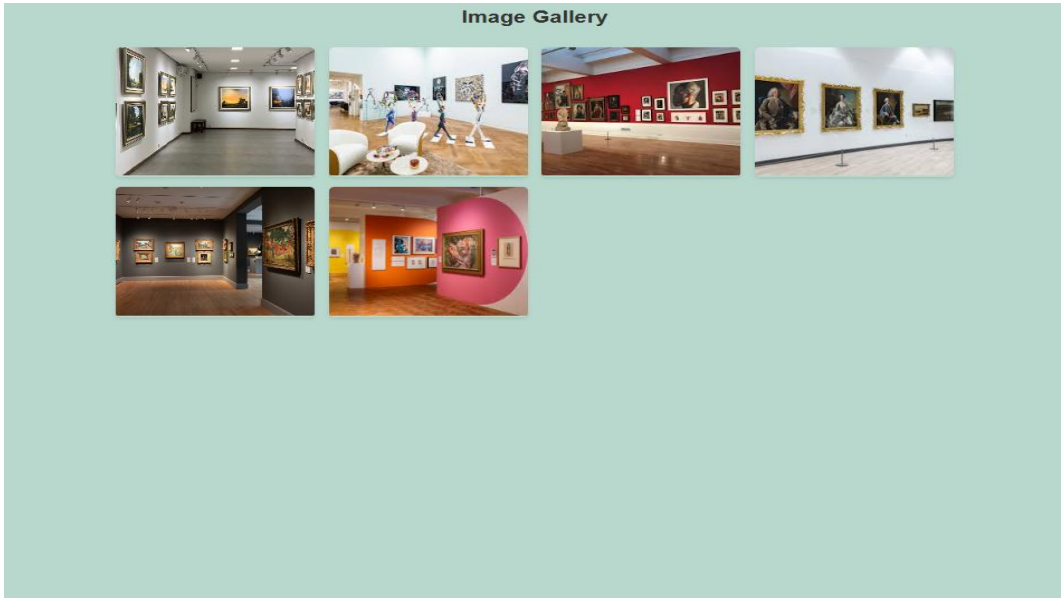
o Ensure mobile responsiveness.

Code:

```html
<!-- Design an Image Gallery Website with HTML and display images in a grid layout with hover zoom or overlay effects. Ensure mobile responsiveness.
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Image Gallery</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: Arial, sans-serif;
            background-color: #b8d8cd;
            padding: 20px;
        }
        h1 {
            text-align: center;
            margin-bottom: 40px;
            color: #333;
        }
        .gallery {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
            gap: 20px;
            max-width: 1200px;
            margin: 0 auto;
        }
        .gallery-item {
            position: relative;
            overflow: hidden;
            border-radius: 8px;
            box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
            cursor: pointer;
        }
        .gallery-item img {
            width: 100%;
            height: 250px;
            object-fit: cover;
            transition: transform 0.3s ease;
        }
        .gallery-item:hover img {
            transform: scale(1.1);
```

```html
<html lang="en">
<head>
    <style>
        }
        .gallery-item:hover img {
            transform: scale(1.1);
        }
        .overlay {
            position: absolute;
            top: 0;
            left: 0;
            width: 100%;
            height: 100%;
            background-color: rgba(0, 0, 0, 0.197);
            opacity: 0;
            transition: opacity 0.3s ease;
            display: flex;
            align-items: center;
            justify-content: center;
            color: rgb(31, 9, 105);
            font-size: 18px;
        }
        .gallery-item:hover .overlay {
            opacity: 1;
        }
    </style>
</head>
<body>
    <h1>Image Gallery</h1>
    <div class="gallery">
        <div class="gallery-item">
            <img src="https://dynamic-media-cdn.tripadvisor.com/media/photo-o/17/56/cd/3d/tanzara-art-gallery.jpg?w=900&h=500&s=1" alt="Image 1">
            <div class="overlay">Image 1</div>
        </div>
        <div class="gallery-item">
            <img src="https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcT_pszMxI9hkBielomkyLmk-ZcG_Ufoga1JcQ&s" alt="Image 2">
            <div class="overlay">Image 2</div>
        </div>
        <div class="gallery-item">
            <img src="https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQKg4A9YzI1IZH00rOOYhxS97TMTufNJbMBDA&s" alt="Image 3">
            <div class="overlay">Image 3</div>
        </div>
        <div class="gallery-item">
            <img src="https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcR3a50Dk8hUkEdVfah5pKgjCR1OYuy9YLrq8Q&s" alt="Image 4">
            <div class="overlay">Image 4</div>
        </div>
        <div class="gallery-item">
            <img src="https://artgallery.yale.edu/sites/default/files/styles/hero_small/public/2023-01/ag-doc-2281-0036-pub.jpg?h=147a4df9&itok=xOjI1bjU" alt="Image 5">
            <div class="overlay">Image 5</div>
        </div>
        <div class="gallery-item">
            <img src=" https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcS1FmCxKDg2_D4jB810WtqnXP5uGYrxt6omkA&s" alt="Image 6">
            <div class="overlay">Image 6</div>
        </div>
```

Output:



Task 2 – Profile Card Generator

Create a profile card web page.

Requirements:

- **Display user image, name, role, and social media links.**
- **Use AI to:**

o Suggest modern UI colors and fonts.

o Design the card using Flexbox.

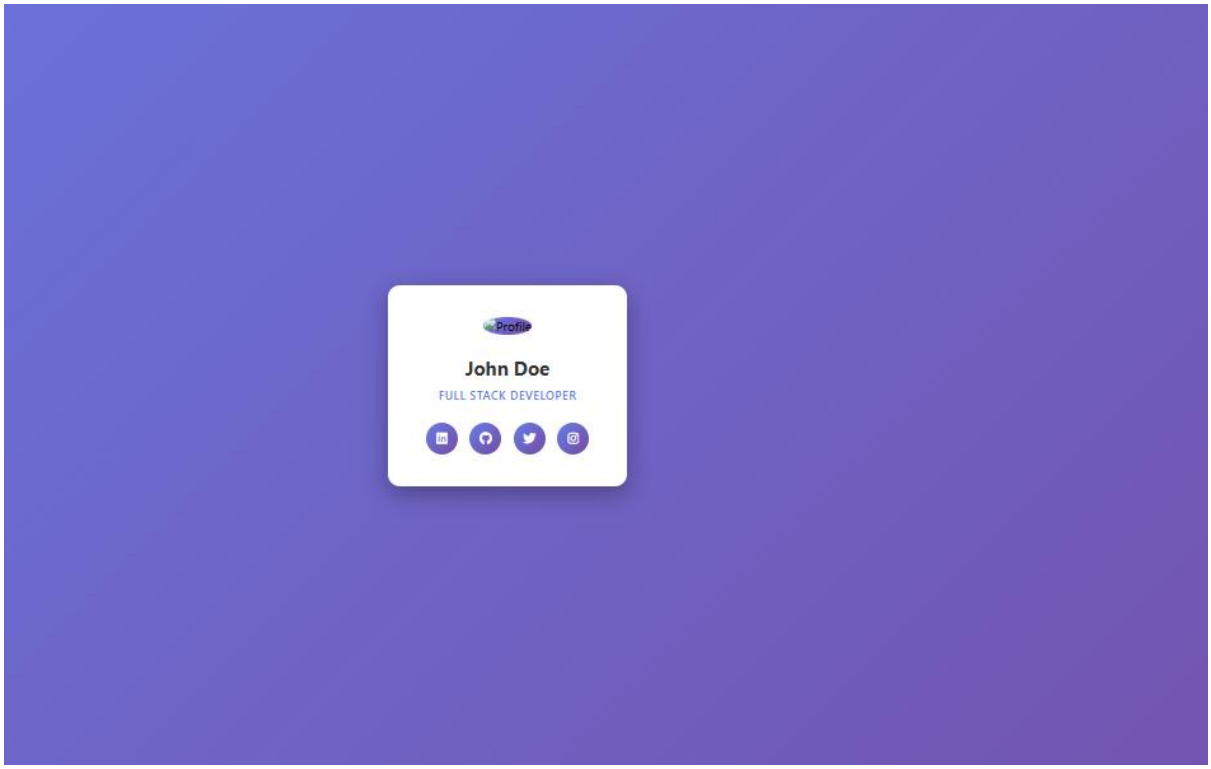
o Add hover animation and shadow effects

Code:

```html
<!-- Design an profile card web page. with HTML and display Display user image, name, role, and social media links by using modern UI colors and fonts ,card using FlexboxAdd hover animation and shadow effects. -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Profile Card</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
            display: flex;
            justify-content: center;
            align-items: center;
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
        }
        .card {
            background: white;
            border-radius: 15px;
            padding: 40px;
            width: 300px;
            text-align: center;
            box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);
            transition: all 0.3s ease;
            cursor: pointer;
        }
        .card:hover {
            transform: translateY(-10px);
            box-shadow: 0 20px 40px rgba(0, 0, 0, 0.4);
        }

        .profile-img {
            width: 120px;
            height: 120px;
            border-radius: 50%;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            margin: 0 auto 20px;
            overflow: hidden;
            object-fit: cover;
        }

        h2 {
            color: #333;
            margin-bottom: 10px;
            font-size: 24px;
        }

        .role {
            color: #667ea
```

```
b-18(2).html > html > head > style > .card
<html lang="en">
<head>
    <style>
        h2 {
            font-size: 24px;
        }

        .role {
            color: #667eea;
            font-size: 14px;
            margin-bottom: 20px;
            font-weight: 600;
            text-transform: uppercase;
            letter-spacing: 1px;
        }

        .social-links {
            display: flex;
            justify-content: center;
            gap: 15px;
            margin-top: 25px;
        }

        .social-links a {
            width: 40px;
            height: 40px;
            border-radius: 50%;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            display: flex;
            justify-content: center;
            align-items: center;
            text-decoration: none;
            transition: all 0.3s ease;
        }

        .social-links a:hover {
            transform: scale(1.2) rotate(10deg);
            box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
        }
    </style>
</head>
<body>
    <div class="card">
        <img src="https://via.placeholder.com/120" alt="Profile" class="profile-img">
        <h2>John Doe</h2>
        <p class="role">Full Stack Developer</p>
        <div class="social-links">
            <a href="#"><i class="fab fa-linkedin"></i></a>
            <a href="#"><i class="fab fa-github"></i></a>
            <a href="#"><i class="fab fa-twitter"></i></a>
            <a href="#"><i class="fab fa-instagram"></i></a>
        </div>
    </div>
```

output:



Task 3 -Develop a dynamic shopping cart system.

Requirements:

- **Add, remove, and update product quantities.**
- **Display real-time total price.**
- **Use AI to:**

o Manage cart state using JavaScript objects.

o Persist cart data in local Storage.

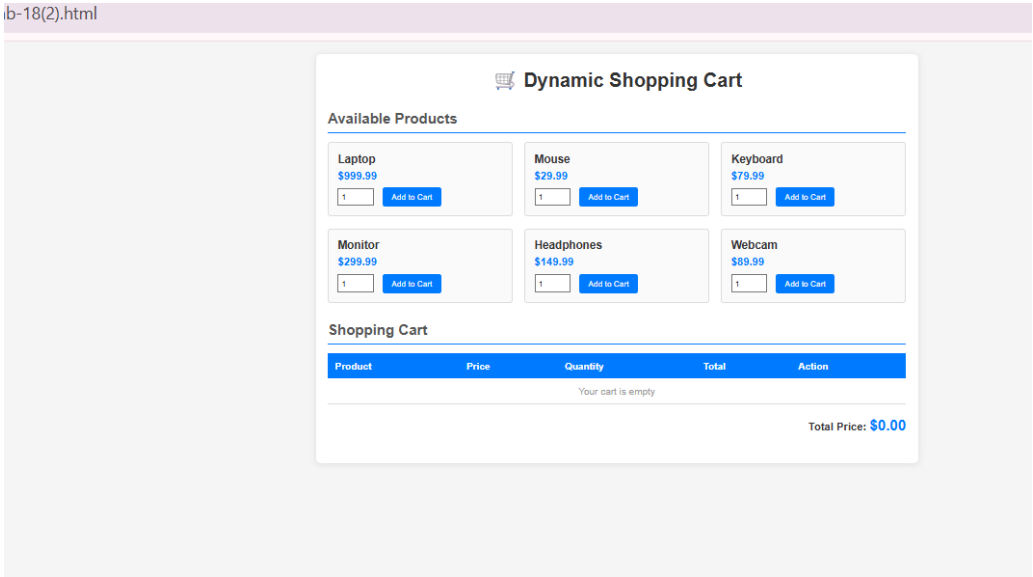
Code:

```html
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dynamic Shopping Cart</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: Arial, sans-serif;
            background-color: #f5f5f5;
            padding: 20px;
        }
        .container {
            max-width: 1000px;
            margin: 0 auto;
            background-color: white;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 2px 10px rgba(0,0,0,0.1);
        }
        h1 {
            text-align: center;
            margin-bottom: 30px;
            color: #333;
        }
        .products-section, .cart-section {
            margin-bottom: 30px;
        }
        h2 {
            color: #555;
            margin-bottom: 15px;
            border-bottom: 2px solid #007bff;
            padding-bottom: 10px;
        }
        .products-grid {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
            gap: 20px;
        }
        .product-card {
            border: 1px solid #ddd;
```

```html
<html lang="en">
<head>
    </style>
</head>
<body>
    <div class="container">
        <h1>🛒 Dynamic Shopping Cart</h1>

        <div class="products-section">
            <h2>Available Products</h2>
            <div class="products-grid" id="productsGrid"></div>
        </div>

        <div class="cart-section">
            <h2>Shopping Cart</h2>
            <table class="cart-items" id="cartTable">
                <thead>
                    <tr>
                        <th>Product</th>
                        <th>Price</th>
                        <th>Quantity</th>
                        <th>Total</th>
                        <th>Action</th>
                    </tr>
                </thead>
                <tbody id="cartBody">
                    <tr><td colspan="5" class="empty-message">Your cart is empty</td>
                </tbody>
            </table>
            <div class="cart-summary">
                Total Price: <span class="total-price" id="totalPrice">$0.00</span>
            </div>
        </div>
    </div>

    <script>
        // Product data
        const products = [
            { id: 1, name: 'Laptop', price: 999.99 },
            { id: 2, name: 'Mouse', price: 29.99 },
            { id: 3, name: 'Keyboard', price: 79.99 },
            { id: 4, name: 'Monitor', price: 299.99 },
            { id: 5, name: 'Headphones', price: 149.99 },
            { id: 6, name: 'Webcam', price: 89.99 }
        ];
```

```
<script>
    function initializeProducts() {
        const grid = document.getElementById('productsGrid');
        products.forEach(product => {
            const card = document.createElement('div');
            card.className = 'product-card';
            card.innerHTML = `
                <h3>${product.name}</h3>
                <div class="product-price">$${product.price.toFixed(2)}</div>
                <input type="number" class="qty-input" value="1" min="1">
                <button onclick="addToCart(${product.id})">Add to Cart</button>
            `;
            grid.appendChild(card);
        });
    }

    // Add product to cart
    function addToCart(productId) {
        const product = products.find(p => p.id === productId);
        const qtyInput = event.target.previousElementSibling;
        const quantity = parseInt(qtyInput.value);

        if (cart[productId]) {
            cart[productId].quantity += quantity;
        } else {
            cart[productId] = {
                name: product.name,
                price: product.price,
                quantity: quantity
            };
        }
        qtyInput.value = 1;
        updateCart();
    }

    // Remove product from cart
    function removeFromCart(productId) {
        delete cart[productId];
        updateCart();
    }

    // Update product quantity
    function updateQuantity(productId) {
        const qtyInput = document.querySelector(`input[data-id="${productId}"]`);
```

Output:



Task 4 – Role-Based Access Control Application

Create a role-based web application.

Requirements:

- **Admin, Editor, User roles.**
- **Conditional UI rendering.**
- **Use AI to:**

o Design access control logic.

o Maintain secure state transitions.

o Create scalable UI components.

Code:

lab-18(2).html > ...

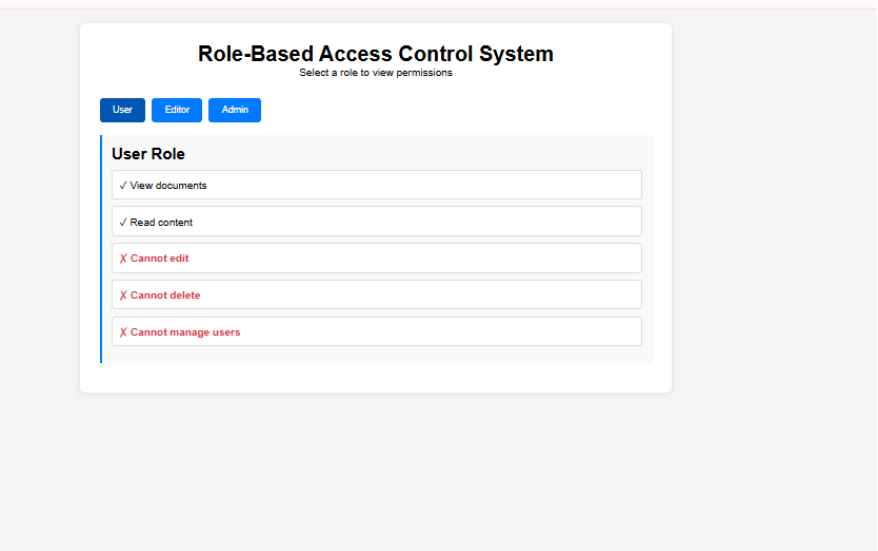
```html
<!-- design a role-based access control system with three roles: User, Editor, and Admin. Each role should have differen
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Role-Based Access Control</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: Arial, sans-serif;
            background: #f5f5f5;
        }
        .container {
            max-width: 900px;
            margin: 20px auto;
            background: white;
            padding: 30px;
            border-radius: 8px;
            box-shadow: 0 2px 10px rgba(0,0,0,0.1);
        }
        .header {
            text-align: center;
            margin-bottom: 30px;
        }
        .role-selector {
            display: flex;
            gap: 10px;
            margin-bottom: 20px;
        }
        button {
            padding: 10px 20px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
            background: #007bff;
            color: white;
            font-size: 14px;
        }
        button.active {
            background: #0056b3;
        }
        .content {
            margin-top: 20px;
        }
        .section {
            display: none;
            padding: 15px;
            background: #f9f9f9;
            border-left: 4px solid #007bff;
```

lab-18(2).html > ...

```html
<html lang="en">
<head>
</head>
<body>
    <div class="container">
        <div class="header">
            <h1>Role-Based Access Control System</h1>
            <p>Select a role to view permissions</p>
        </div>

        <div class="role-selector">
            <button class="role-btn active" data-role="user">User</button>
            <button class="role-btn" data-role="editor">Editor</button>
            <button class="role-btn" data-role="admin">Admin</button>
        </div>

        <div class="content">
            <div id="user-section" class="section active">
                <h2>User Role</h2>
                <div class="feature">✓ View documents</div>
                <div class="feature">✓ Read content</div>
                <div class="feature restricted">✗ Cannot edit</div>
                <div class="feature restricted">✗ Cannot delete</div>
                <div class="feature restricted">✗ Cannot manage users</div>
            </div>

            <div id="editor-section" class="section">
                <h2>Editor Role</h2>
                <div class="feature">✓ View documents</div>
                <div class="feature">✓ Edit documents</div>
                <div class="feature">✓ Create documents</div>
                <div class="feature restricted">✗ Cannot delete</div>
                <div class="feature restricted">✗ Cannot manage users</div>
            </div>

            <div id="admin-section" class="section">
                <h2>Admin Role</h2>
                <div class="feature">✓ View all documents</div>
                <div class="feature">✓ Edit all documents</div>
                <div class="feature">✓ Create documents</div>
                <div class="feature">✓ Delete documents</div>
                <div class="feature">✓ Manage users</div>
            </div>
        </div>
    </div>

    <script>
        class AccessControl {
            constructor() {
                this.currentRole = 'user';
                this.initEventListeners();
            }

            initEventListeners() {
                document.querySelectorAll('.role-btn').forEach(btn => {
```

```html
                <div class="feature">✓ Manage users</div>
            </div>
        </div>
    </div>

    <script>
        class AccessControl {
            constructor() {
                this.currentRole = 'user';
                this.initEventListeners();
            }

            initEventListeners() {
                document.querySelectorAll('.role-btn').forEach(btn => {
                    btn.addEventListener('click', (e) => this.switchRole(e.target.dataset.role));
                });
            }

            switchRole(role) {
                this.currentRole = role;
                this.updateUI();
            }

            updateUI() {
                document.querySelectorAll('.role-btn').forEach(btn => {
                    btn.classList.toggle('active', btn.dataset.role === this.currentRole);
                });

                document.querySelectorAll('.section').forEach(section => {
                    section.classList.remove('active');
                });

                document.getElementById(`${this.currentRole}-section`).classList.add('active');
            }
        }

        new AccessControl();
    </script>
</body>
</html>
```

Output:



Task 5 – Multi-Language Website

Design a multilingual website.

Requirements:

- **Switch languages dynamically.**

- **Persist language preference.**
- **Use AI to:**

o Structure translation files.

o Handle RTL/LTR layouts.

o Improve accessibility.

Code:

```html
<!-- design a esign a multilingual website . Switch languages dynamicallyPersist la
Structure translation files.Handle RTL/LTR layouts.Improve accessibility. -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Multilingual Website</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: Arial, sans-serif;
            transition: direction 0.3s;
        }
        body[dir="rtl"] {
            direction: rtl;
            text-align: right;
        }
        body[dir="ltr"] {
            direction: ltr;
            text-align: left;
        }
        header {
            background: #333;
            color: white;
            padding: 20px;
            display: flex;
            justify-content: space-between;
            align-items: center;
        }
        .language-selector {
            display: flex;
            gap: 10px;
```

```html
<head>
    <style>
        button {
            padding: 8px 12px;
            cursor: pointer;
            border: none;
            background: #0066cc;
            color: white;
            border-radius: 4px;
        }
        button.active {
            background: #ff9900;
        }
        main {
            padding: 40px;
            max-width: 800px;
            margin: 0 auto;
        }
        h1, h2 {
            margin-bottom: 20px;
        }
        p {
            line-height: 1.6;
            margin-bottom: 15px;
        }
        footer {
            background: #333;
            color: white;
            text-align: center;
            padding: 20px;
            margin-top: 40px;
        }
    </style>
</head>
<body dir="ltr">
    <header>
        <h1 data-i18n="title">Multilingual Website</h1>
        <div class="language-selector">
            <button data-language="en" class="active" aria-label="English">EN</button>
            <button data-language="es" aria-label="Español">ES</button>
            <button data-language="ar" aria-label="العربية">AR</button>
        </div>
    </header>

    <main>
        <h2 data-i18n="welcome">Welcome</h2>
        <p data-i18n="description">This is a multilingual website with dynamic language switching.</p>
        <h2 data-i18n="features">Features</h2>
        <ul>
            <li data-i18n="feature1">Dynamic language switching</li>
            <li data-i18n="feature2">Persistent language preference</li>
            <li data-i18n="feature3">RTL and LTR support</li>
            <li data-i18n="feature4">Improved accessibility</li>
        </ul>
```

```html
<html lang="en">
<head>
</head>
<body dir="ltr">
    <header>
        <h1 data-i18n="title">Multilingual Website</h1>
        <div class="language-selector">
            <button data-language="en" class="active" aria-label="English">EN</button>
            <button data-language="es" aria-label="Español">ES</button>
            <button data-language="ar" aria-label="العربية">AR</button>
        </div>
    </header>

    <main>
        <h2 data-i18n="welcome">Welcome</h2>
        <p data-i18n="description">This is a multilingual website with dynamic language switching
        <h2 data-i18n="features">Features</h2>
        <ul>
            <li data-i18n="feature1">Dynamic language switching</li>
            <li data-i18n="feature2">Persistent language preference</li>
            <li data-i18n="feature3">RTL and LTR support</li>
            <li data-i18n="feature4">Improved accessibility</li>
        </ul>
    </main>

    <footer>
        <p data-i18n="footer">&copy; 2024 Multilingual Website</p>
    </footer>

    <script>
        const translations = {
            en: {
                title: "Multilingual Website",
                welcome: "Welcome",
                description: "This is a multilingual website with dynamic language switching.",
                features: "Features",
                feature1: "Dynamic language switching",
                feature2: "Persistent language preference",
                feature3: "RTL and LTR support",
                feature4: "Improved accessibility",
                footer: "© 2024 Multilingual Website"
            },
            es: {
                title: "Sitio Web Multilingüe",
                welcome: "Bienvenido",
                description: "Este es un sitio web multilingüe con cambio de idioma dinámico.",
                features: "Características",
                feature1: "Cambio de idioma dinámico",
                feature2: "Preferencia de idioma persistente",
                feature3: "Soporte RTL y LTR",
                feature4: "Accesibilidad mejorada",
                footer: "© 2024 Sitio Web Multilingüe"
            },
```

Output:

Task 6 – Online Examination System

Create an online exam interface.

Requirements:

- **Timer, questions, and submission.**
- **Prevent multiple submissions.**
- **Use AI to:**

o Manage exam state.

o Improve exam UX.

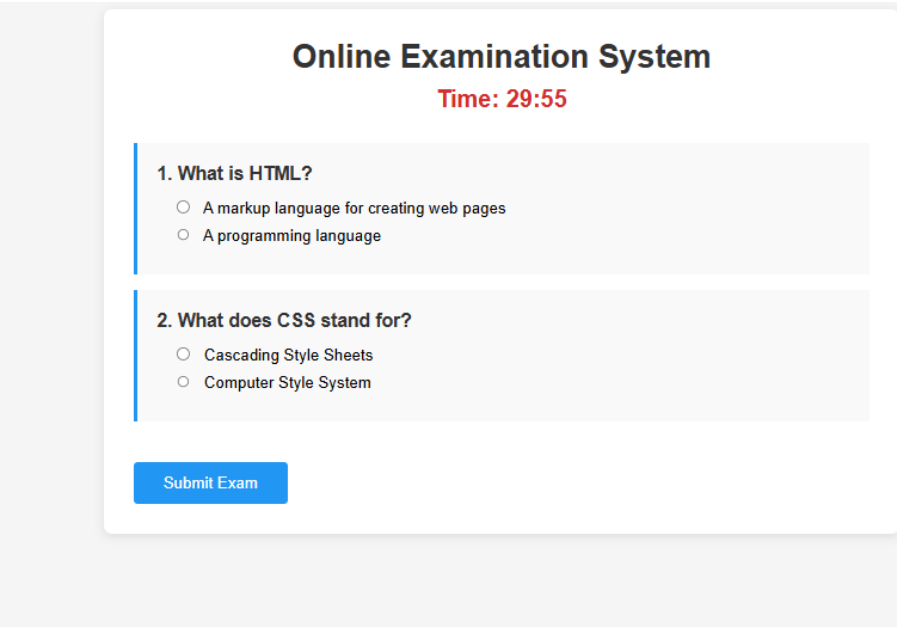
o Ensure accessibility.

Code :

lab-18(2).html > html > head > style > input[type="radio"]

```html
<!-- design a Online Examination System.Timer, questions, and submission.Prevent multiple submissions.with html , manage exam state. ensure asscebilty  -->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Online Examination System</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .container { max-width: 800px; margin: 20px auto; background: white; padding: 30px; border-radius: 8px; box-shadow: 0 2px 10px rgba(0,0,0,0.1); }
        header { text-align: center; margin-bottom: 30px; }
        h1 { color: #333; margin-bottom: 10px; }
        .timer { font-size: 24px; font-weight: bold; color: #d32f2f; }
        .exam-section { margin: 20px 0; }
        .question { background: #f9f9f9; padding: 20px; margin: 15px 0; border-left: 4px solid #2196F3; }
        .question h3 { margin-bottom: 15px; color: #333; }
        .options { margin-left: 20px; }
        .option { margin: 10px 0; }
        input[type="radio"] { margin-right: 10px; cursor: pointer; }
        label { cursor: pointer; }
        button { background: #2196F3; color: white; padding: 12px 30px; border: none; border-radius: 4px; font-size: 16px; cursor: pointer; margin-top: 20px; }
        button:hover { background: #1976D2; }
        button:disabled { background: #ccc; cursor: not-allowed; }
        .status { padding: 15px; border-radius: 4px; margin-bottom: 20px; }
        .success { background: #4CAF50; color: white; }
        .error { background: #d32f2f; color: white; }
        @media (max-width: 600px) { .container { padding: 15px; } .question { padding: 15px; } }
    </style>
</head>
<body>
    <div class="container">
        <header>
            <h1>Online Examination System</h1>
            <div class="timer" aria-label="Time remaining" role="status">Time: <span id="timeDisplay">30:00</span></div>
        </header>

        <div id="statusMessage"></div>

        <form id="examForm" aria-label="Exam questions form">
            <div class="exam-section">
                <div class="question">
                    <h3>1. What is HTML?</h3>
                    <div class="options">
                        <div class="option">
                            <input type="radio" id="q1a" name="q1" value="a">
                            <label for="q1a">A markup language for creating web pages</label>
                        </div>
                        <div class="option">
                            <input type="radio" id="q1b" name="q1" value="b">
                            <label for="q1b">A programming language</label>
                        </div>
                    </div>
```

```html
<body>
    <div class="container">
        <form id="examForm" aria-label="Exam questions form">
            <div class="exam-section">
                <div class="option">
                    <input type="radio" id="q2a" name="q2" value="a">
                    <label for="q2a">Cascading Style Sheets</label>
                </div>
                <div class="option">
                    <input type="radio" id="q2b" name="q2" value="b">
                    <label for="q2b">Computer Style System</label>
                </div>
            </div>
        </div>
    </div>
    <button type="submit" id="submitBtn" aria-label="Submit exam">Submit Exam</button>
    </form>
</div>
<script>
    const EXAM_DURATION = 30 * 60; // 30 minutes in seconds
    let timeLeft = EXAM_DURATION;
    let examSubmitted = false;

    // Timer functionality
    function startTimer() {
        const timerInterval = setInterval(() => {
            if (timeLeft <= 0) {
                clearInterval(timerInterval);
                autoSubmitExam();
                return;
            }
            timeLeft--;
            updateTimerDisplay();

            if (timeLeft === 60) {
                alert('⏰ Warning: 1 minute remaining!');
            }
        }, 1000);
    }
    function updateTimerDisplay() {
        const minutes = Math.floor(timeLeft / 60);
        const seconds = timeLeft % 60;
        document.getElementById('timeDisplay').textContent =
            `${minutes}:${seconds < 10 ? '0' : ''}${seconds}`;
    }
    // Form submission
    document.getElementById('examForm').addEventListener('submit', (e) => {
        e.preventDefault();

        if (examSubmitted) {
            showStatus('Exam already submitted!', 'error');
            return;
        }
```

```javascript
        const timerInterval = setInterval(() => {

            if (timeLeft === 60) {
                alert('⏰ Warning: 1 minute remaining!');
            }
        }, 1000);
    }
    function updateTimerDisplay() {
        const minutes = Math.floor(timeLeft / 60);
        const seconds = timeLeft % 60;
        document.getElementById('timeDisplay').textContent =
            `${minutes}:${seconds < 10 ? '0' : ''}${seconds}`;
    }
    // Form submission
    document.getElementById('examForm').addEventListener('submit', (e) => {
        e.preventDefault();

        if (examSubmitted) {
            showStatus('Exam already submitted!', 'error');
            return;
        }
        submitExam();
    });

    function submitExam() {
        examSubmitted = true;
        document.getElementById('submitBtn').disabled = true;
        document.getElementById('submitBtn').textContent = 'Exam Submitted';
        showStatus('✓ Your exam has been submitted successfully!', 'success');

        // Prevent form changes after submission
        const inputs = document.querySelectorAll('input[type="radio"]');
        inputs.forEach(input => input.disabled = true);
    }

    function autoSubmitExam() {
        if (!examSubmitted) {
            submitExam();
            showStatus('⏱ Time is up! Your exam has been auto-submitted.', 'error');
        }
    }

    function showStatus(message, type) {
        const statusDiv = document.getElementById('statusMessage');
        statusDiv.textContent = message;
        statusDiv.className = `status ${type}`;
    }

    // Initialize
```

Output :